

Representation and Description

Introduction

- After an image has been segmented into regions, the resulting aggregate of segmented pixels usually is represented and described in a form suitable for further processing.
- Representing a region involves two choices:
 - I. Represent regions in terms of its external characteristics (its boundary), or
 - II. Represent regions in terms of its internal characteristics (pixels comprising the region)
- An **external representation** is chosen when the primary focus is on shape characteristics.
- An **internal representation** is chosen when the primary focus is on regional properties, such as color and texture.
- The next step is to *describe* the region based on the chosen representation.
- For example: a region may be represented by its boundary, and its boundary described by it some features such as length, orientation etc.
- Features should be insensitive to translation, rotation, and scaling.
- Both *boundary* and *regional descriptors* are often used together.

Types

- Representations can be classified into following two types:
 1. Chain Codes
 2. Signatures

Chain Codes

- Chain codes are used to represent a boundary by a connected sequence of straight-line segments of specified length and direction.
- Typically, this representation is based on 4- or 8- connectivity of the segments.
- The direction of each segment is coded by using a numbering scheme, as shown in figure below.
- A boundary code formed as a sequence of such directional numbers is referred to as *Freeman chain code*.

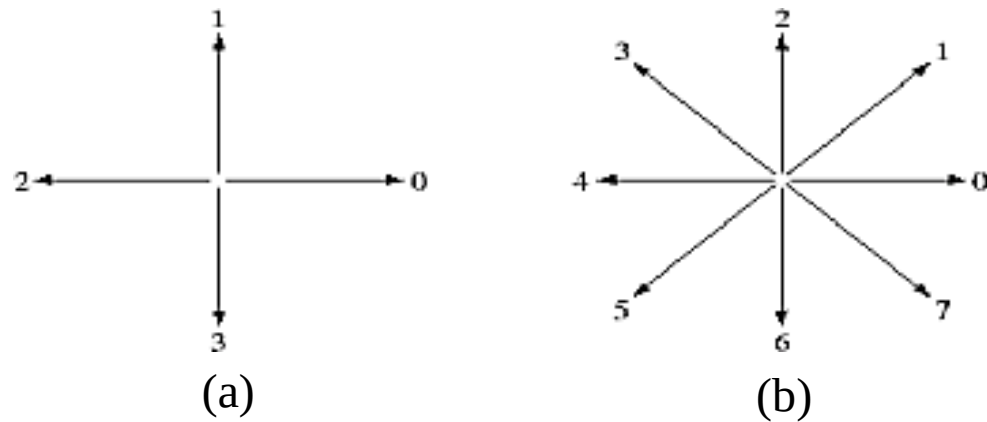


Figure: Directional numbers for
(a) 4-directional chain code, and
(b) 8-directional chain code

Chain Codes

- The method generally is unacceptable for two principal reasons:
 1. The resulting chain of codes tends to be quite long and
 2. Any small disturbances cause changes in the code that may not be related to the shape of the boundary.
- An approach frequently used to circumvent the problem just discussed is to resample the boundary by selecting a larger grid spacing, as shown in figure aside.

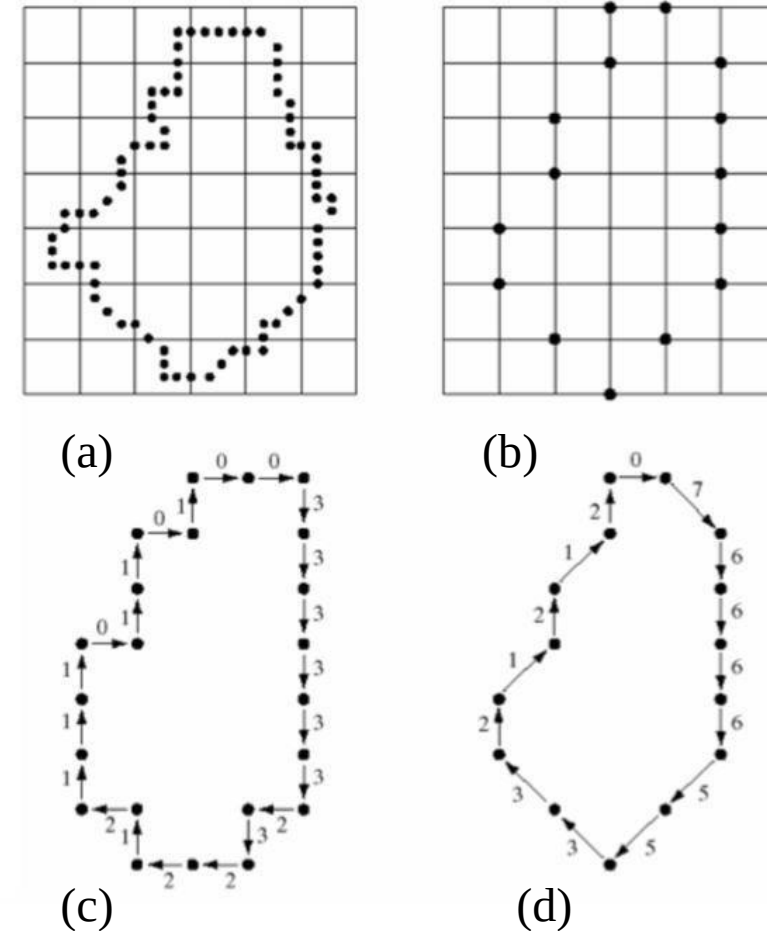


Figure: (a) Digital boundary with resampling grid superimposed. (b) Result of resampling (c) 4-directional code. (d) 8-directional chain code

Chain code

- The chain code of a boundary depends on the starting point. However, the code can be normalized with respect to the starting point by treating it as a circular sequence of direction numbers and redefining the starting point so that the resulting sequence of numbers is a integer magnitude.
- We can normalize also for rotation by using the *first difference* of the chain code instead of the code itself. The difference is obtained by counting the number of direction changes.
- For example: the *first difference* of the 4-direction chain code 10103322 is 3133030.
- If we treat the code as a circular sequence to normalize with respect to the starting point, then first element of the difference is computed by using the transition between the last and first components of the chain.
- Here the result is 33133030.
- The first difference of smallest magnitude (i.e. obtained by treating the resulting array as a circular array and rotating it cyclically until the resulting numerical pattern results is the smallest possible number) is known as *shape number* of the counter.

Chain code

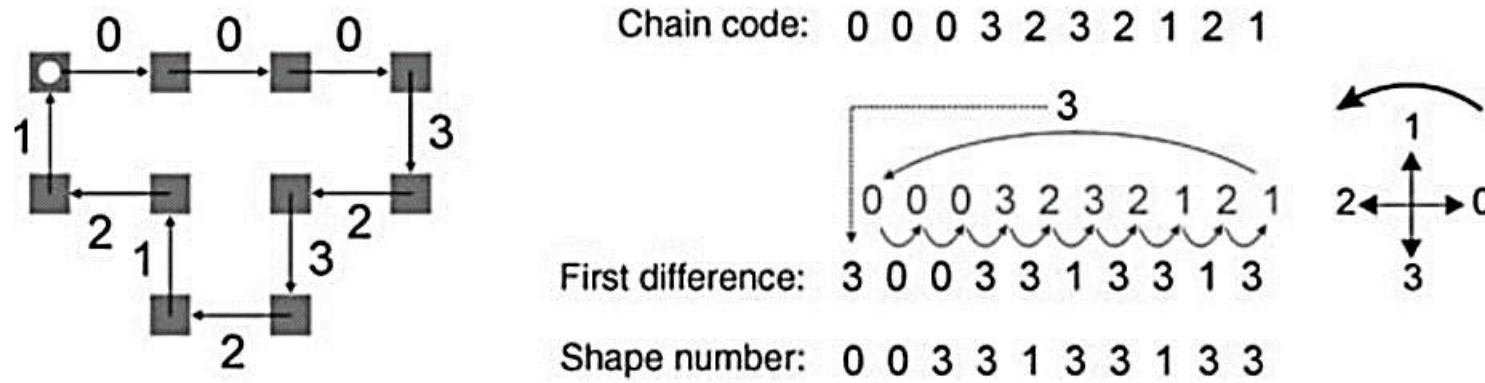
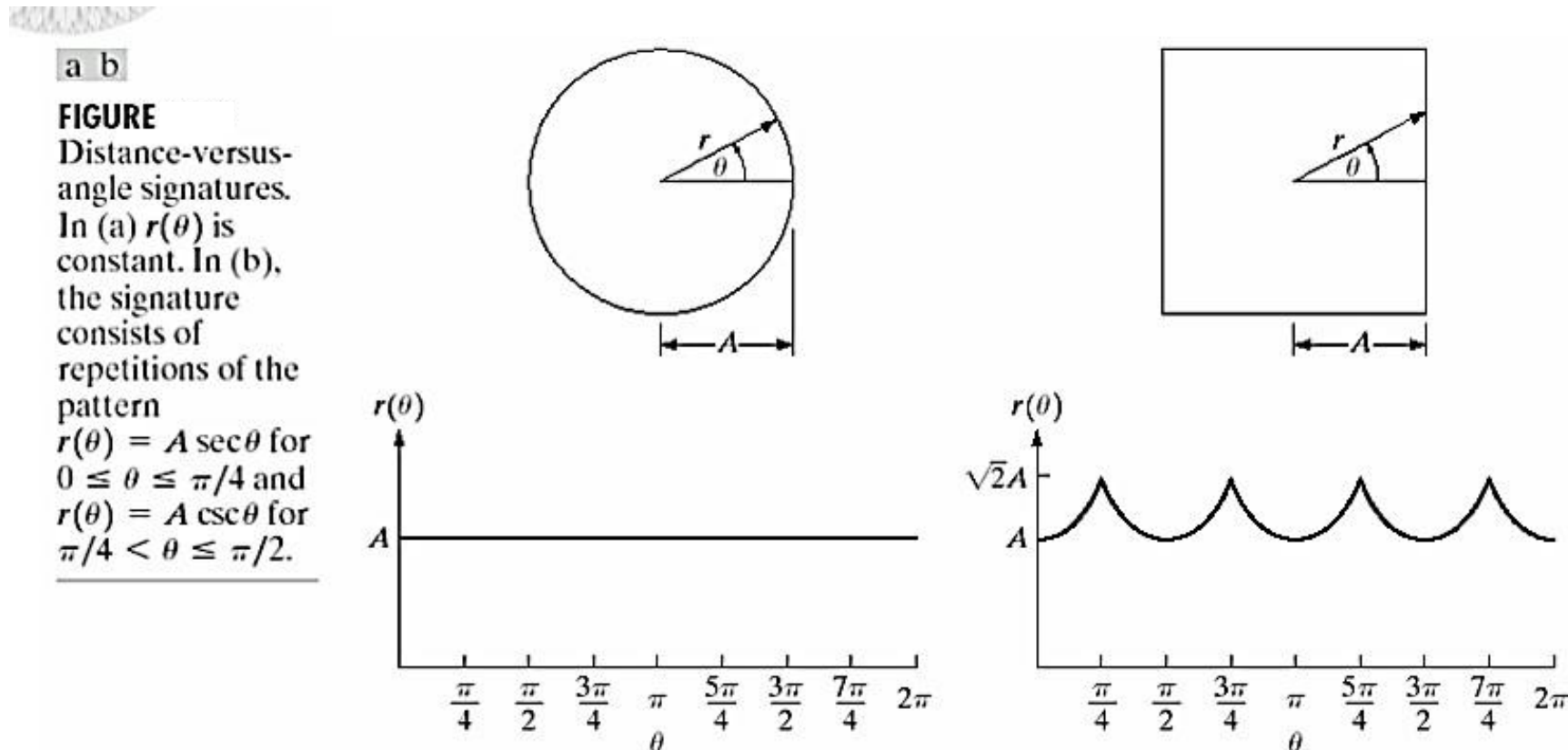


Figure: Chain code, first difference, and shape number

Signatures

- Signatures is a 1-D functional representation of a boundary and may be generated in various ways.
- One of the simplest method is plot the distance from the centroid to the boundary as a function of angle, as illustrated in figure below.



Signatures

- Signatures generated by the approach just described are invariant to translation, but they do depend on rotation and scaling.
- Normalization with respect to rotation can be achieved by finding a way to select the same starting point to generate the signature, regardless of the shape's orientation.
- One way to normalize for this result is to scale all functions so that they always span the same range of values, say, $[0, 1]$
- Whatever the method used, keep in mind that the basic idea is to remove dependency on size while preserving the fundamental shape of the waveforms.

Signatures

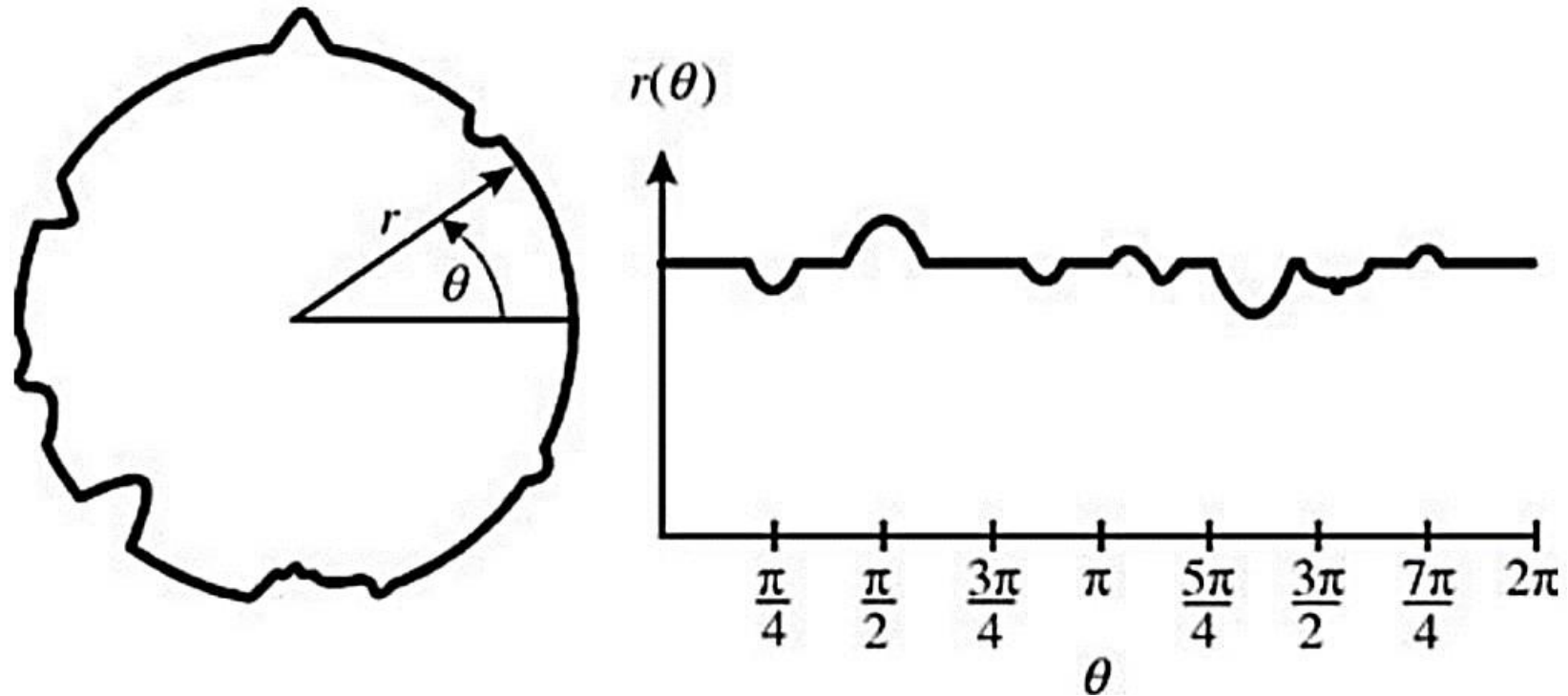


Figure: Effect of noise on signatures for two different objects.

Descriptors

- Approaches to describe the boundary of a region.
- Types
 - Boundary Descriptors
 - Regional Descriptors
 - Relational Descriptors

Some simple Descriptors

- The length of a boundary is one of its simplest descriptors.
- The number of pixels along a boundary gives rough approximation of its length.
- The diameter of a boundary is defined as

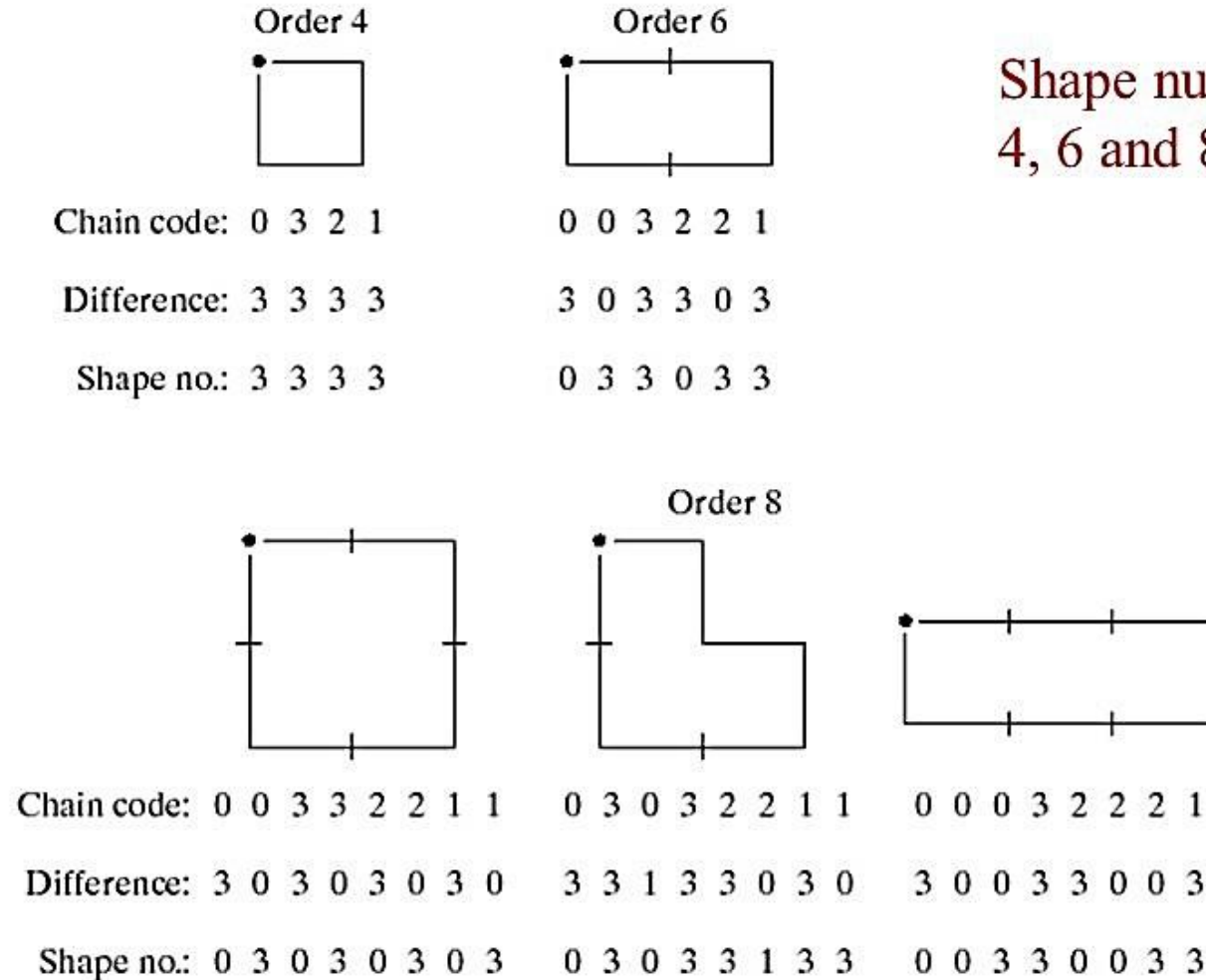
$$Diam(B) = \max_{i,j} [D(p_i, p_j)]$$
 where D is a distance measure and p_i and p_j are points on boundary.

- The line segment connecting the two extreme points that comprise the diameter is called the *major axis* of the boundary and the line perpendicular to the major axis is called *minor axis*.

Shape Number

- The shape number of boundary generally based on four directional chain codes is defined as the first difference of smallest magnitude.
- The *order n* of a shape number is defined as the number of digits in its representation.
-

Shape Number (cont.)

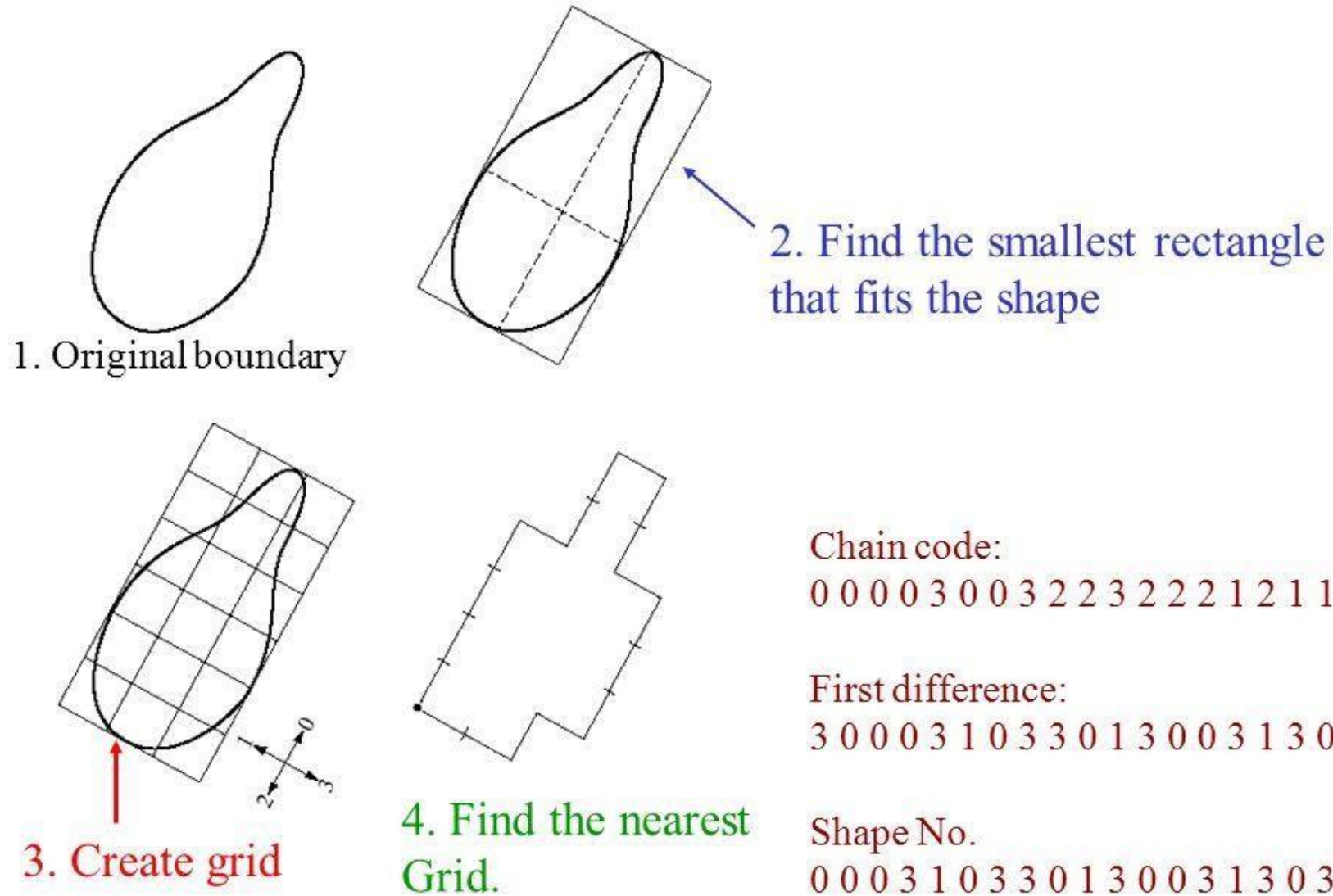


Shape numbers of order 4, 6 and 8

Figure: shape number of order 4, 6 and 8 along with their chain-code representations, first difference, and corresponding shape numbers.

(Images from Rafael C. Gonzalez and Richard E. Wood, Digital Image Processing, 2nd Edition.

Example: Shape Number



(Images from Rafael C. Gonzalez and Richard E. Wood, Digital Image Processing, 2nd Edition.

Example: suppose $n=18$ is specified for the boundary.

- To obtain the shape number of this order, following steps are done:
 1. Find the basic rectangle.
 2. The closet rectangle of order 18 is 3×6 rectangle; sub-divide the rectangle.
 3. Align the chain code directions with the resulting grid.
 4. Obtain the chain code and use its first difference to compute the shape number.

Fourier Descriptors

- The idea behind Fourier Descriptors is to traverse the pixels belonging to a boundary starting from an arbitrary point, and record their coordinates.
- Each value in the resulting list of coordinates pairs (x_0, y_0) , (x_1, y_1) , ..., (x_{k-1}, y_{k-1}) is then interpreted as a complex number $x_k + jy_k$, for $k = 0, 1, \dots, K-1$.
- The discrete Fourier transform (DFT) of this list of complex numbers is the Fourier descriptor of the boundary.
- The inverse DFT restores the original boundary.
- Following figure shows a K-point digital boundary in the xy plane and the first two coordinates pairs (x_0, y_0) and (x_1, y_1) .

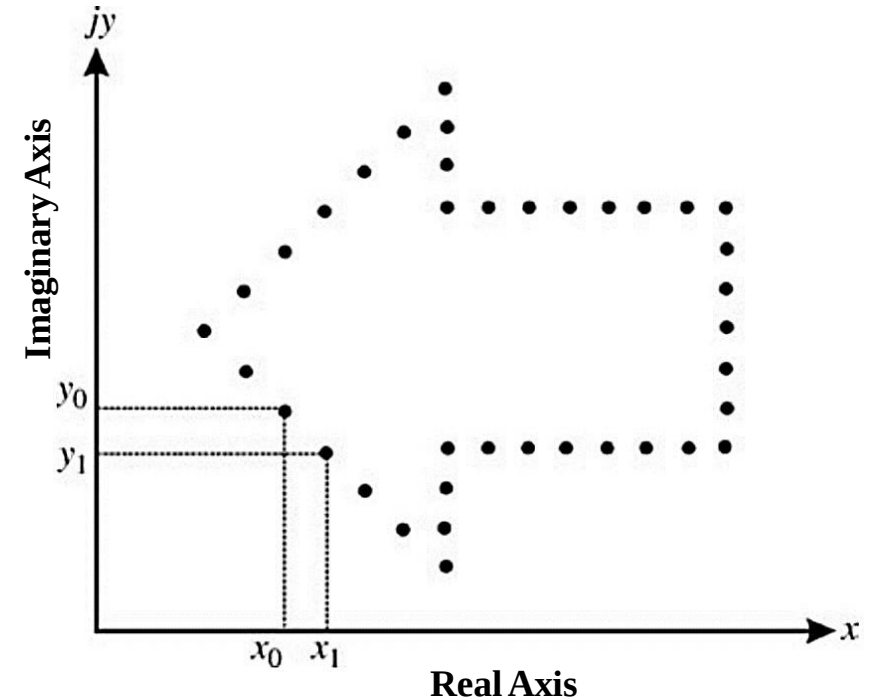


Figure: Fourier Descriptor of a boundary

Steps for Fourier Descriptors

- Define the arbitrary point (x_0, y_0) and the coordinate point or pairs $(x_0, y_0), (x_1, y_1), \dots, (x_{K-1}, y_{K-1})$ are encountered in traversing the boundary in counter clockwise direction.
- These coordinates can be expressed in the form $x(k) = x_k$ and $y(k) = y_k$. The boundary can be represented as the sequence of coordinates as:

$$s(k) = [x(k), y(k)] \text{ for } k = 0, 1, 2, \dots, K-1.$$

The each coordinates pair can be treated as a complex number so that:

$$s(k) = x(k) + jy(k).$$

- The discrete Fourier transform of 1-D sequence, $s(k)$ can be written as:

$$a(u) = \frac{1}{K} \sum_{k=0}^{K-1} s(k) e^{-j2\pi uk/K} \quad \text{where, } u = 0, 1, 2, \dots, K-1$$

- The complex coefficient $a(u)$ are called the Fourier descriptors of the boundary.
- The inverse Fourier transform of these coefficient is:

$$s(k) = \frac{1}{K} \sum_{u=0}^{K-1} a(u) e^{j2\pi uk/K} \quad \text{where, } k = 0, 1, 2, \dots, K-1$$

Pattern Recognition

Human Perception

- Humans have developed highly sophisticated skills for sensing their environment and taking actions according to what they observe. Examples:
 - Recognizing a face,
 - Understanding spoken words,
 - Reading handwriting,
 - Distinguishing fresh food from its smell. Etc.
- We would like to give similar capabilities to machines.

What is pattern?

- A *pattern* is an entity, vaguely defined, that could be given a name , e.g.,
 - Fingerprint image
 - Handwritten word
 - Human face
 - Speech signal
 - DNA Sequence,
 -
- A pattern in *an arraignment of descriptors*. The name feature is often used in to denote a descriptor.

What is pattern recognition?

- A **pattern class** is a family pattern that share some common properties. Pattern classes are denoted by $w_1, w_2, w_3, \dots, w_W$ where W is the number of classes.
- A **pattern recognition/pattern classification** is the study of how machines can
 - Observe the environment,
 - Learn to distinguish patterns of interest,
 - Make sound and reasonable decisions about the categories of the patterns.
- Pattern recognition by machines *involves for assigning patterns to their respective classes* – automatically and with as little human intervention as possible.

- Pattern vectors are represented by bold lowercase letters, such as **x**, **y**, and **z**, and take the form

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdot \\ \cdot \\ \cdot \\ x_n \end{bmatrix} \quad \text{Equivalently, } \mathbf{x} = (x_1, x_2, x_3, \dots, x_n)^T$$

- Where each components, $\mathbf{x}_i$, represent the i^{th} descriptor and n is the total number of such descriptors associated with the pattern.

Goal of Pattern Recognition

- To assign a class to each image (or object within an image) based on a numerical representation of the image's (or object's properties).

Techniques:

- Pattern classification techniques are usually classified into two main groups:
 1. Statistical, and
 2. Structural (or syntactical)

Pattern Recognition Application

Table 1: Example pattern recognition applications.

Problem Domain	Application	Input Pattern	Pattern Classes
Document image analysis	Optical character recognition	Document image	Characters, words
Document classification	Internet search	Text document	Semantic categories
Document classification	Junk mail filtering	Email	Junk/non-junk
Multimedia database retrieval	Internet search	Video clip	Video genres
Speech recognition	Telephone directory assistance	Speech waveform	Spoken words
Natural language processing	Information extraction	Sentences	Parts of speech
Biometric recognition	Personal identification	Face, iris, fingerprint	Authorized users for access control
Medical	Computer aided diagnosis	Microscopic image	Cancerous/healthy cell
Military	Automatic target recognition	Optical or infrared image	Target type
Industrial automation	Printed circuit board inspection	Intensity or range image	Defective/non-defective product
Industrial automation	Fruit sorting	Images taken on a conveyor belt	Grade of quality
Remote sensing	Forecasting crop yield	Multispectral image	Land use categories
Bioinformatics	Sequence analysis	DNA sequence	Known types of genes
Data mining	Searching for meaningful patterns	Points in multidimensional space	Compact and well-separated clusters

An Example

- **Problem:** Sorting incoming fish on conveyor belt according to species.
- Assume that we have only two kinds of fish:
 - Sea bass,
 - salmon



Figure: Picture Taken from a camera

An Example: Decision Process

- What kind of information can distinguish one species from the other?
 - Length, width, number and shape of fins, tail shape etc.
- What can cause problems during sensing?
 - Lighting conditions, position of fish on the conveyor belt, camera noise, etc.
- What are the steps in the process?
 - Capture image → isolate fish → take measurement → make decision

An Example : Selecting Features

- Assume a fisherman told us that a sea bass is generally longer than a salmon.
- We can use length as a *feature* and decide between sea bass and salmon according to a threshold on length.
- How can we choose this threshold?

An Example : Selecting Features

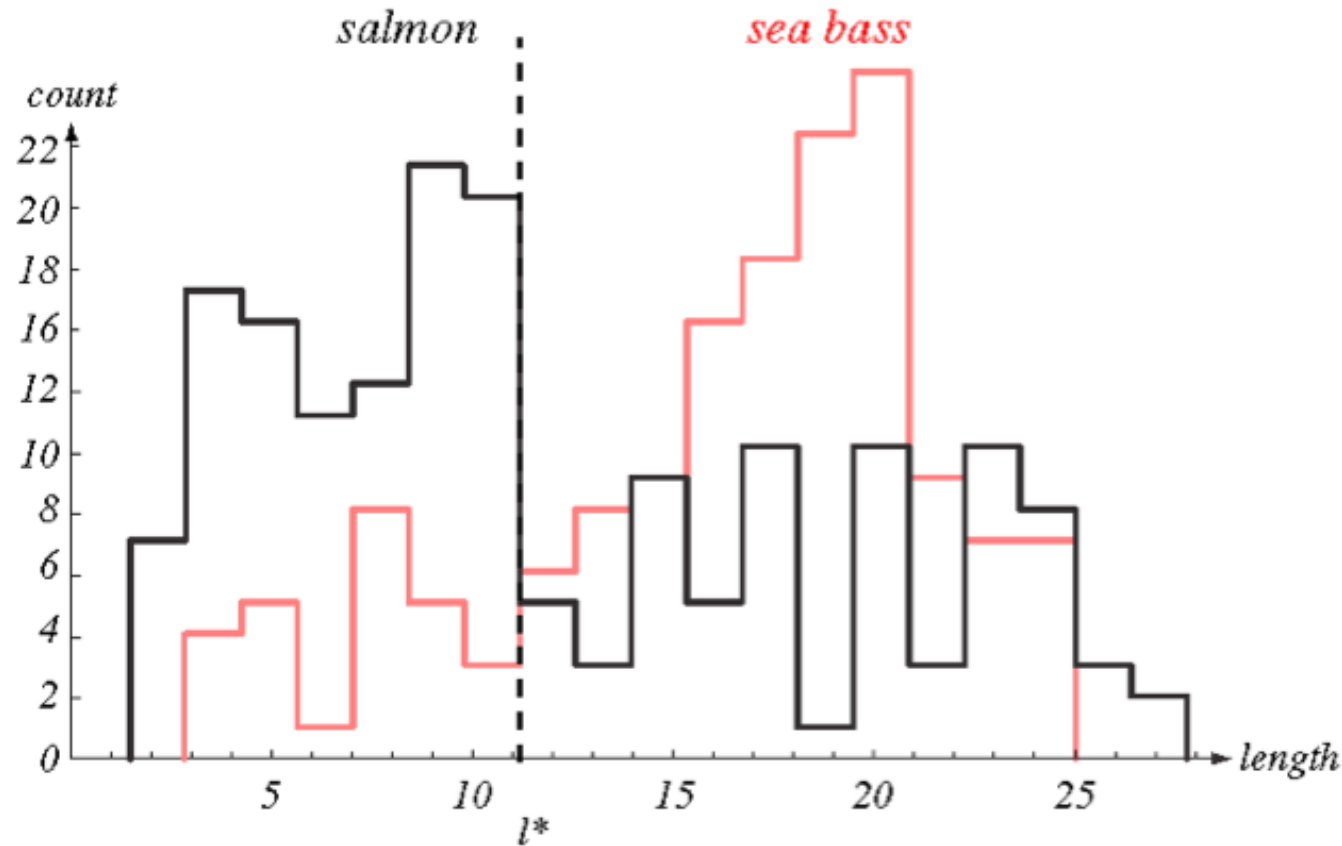


Figure 2: *Histograms* of the length feature for two types of fish in *training samples*. How can we choose the threshold l^* to make a reliable decision?

An Example : Selecting Features

- Even though sea bass is longer than salmon on the average, there are many examples of fish where this observation does not hold.
- Try another feature, average lightness of the fish scales.

An Example : Selecting Features

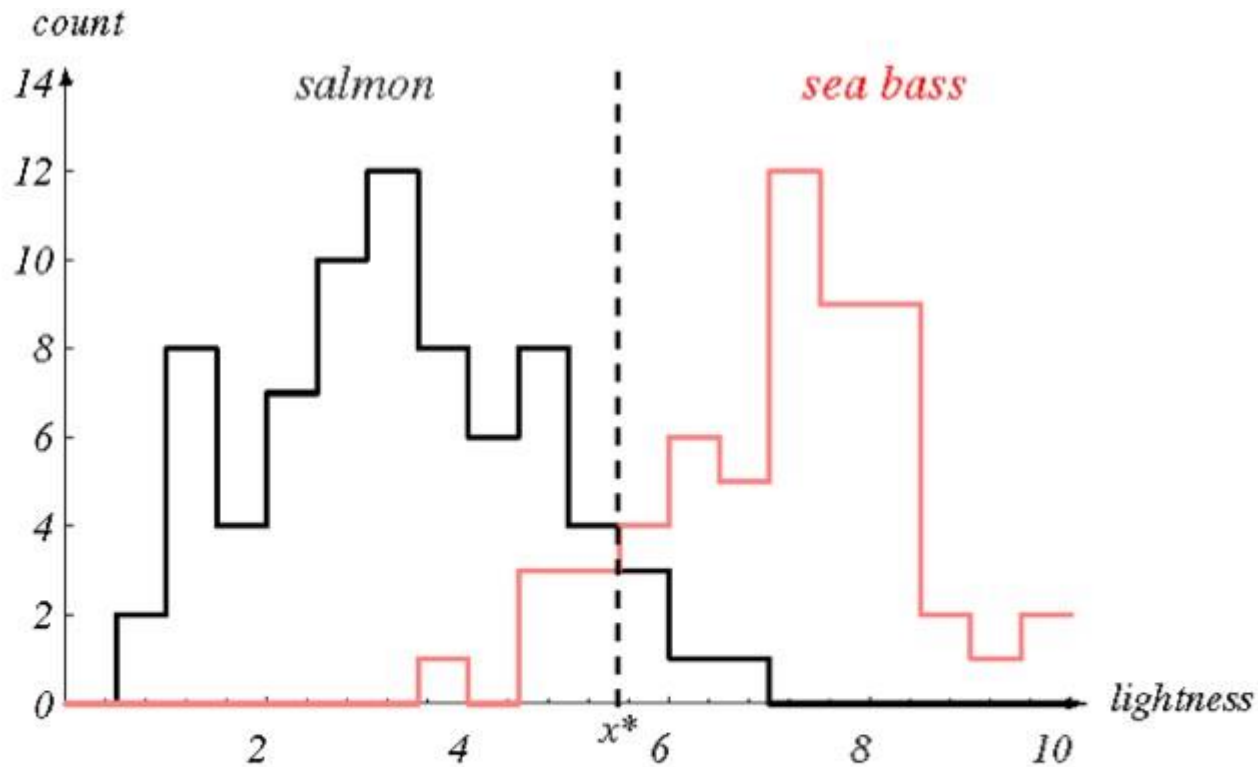


Figure 3: Histograms of the lightness feature for two types of fish in training samples. It looks easier to choose the threshold x^* but we still cannot make a perfect decision.



An Example: Cost of Error

- We should also consider *costs of different errors* we make in our decisions.
- For example, if the fish packing company knows that:
 - Customers who buy salmon will object vigorously if they see sea bass in their cans.
 - Customers who buy sea bass will not be unhappy, if they occasionally see some expensive salmon in their cans.
- How does this knowledge affect our decision?

An Example: Cost of Error

- Assume we also observed that sea bass are typically wider than salmon.
- We can use two features in our decision:
 - Lightness: x_1
 - Width : x_2
- Each fish image is now represented as a point (*feature vector*)

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

in a two-dimensional feature space.

An Example: Multiple Features

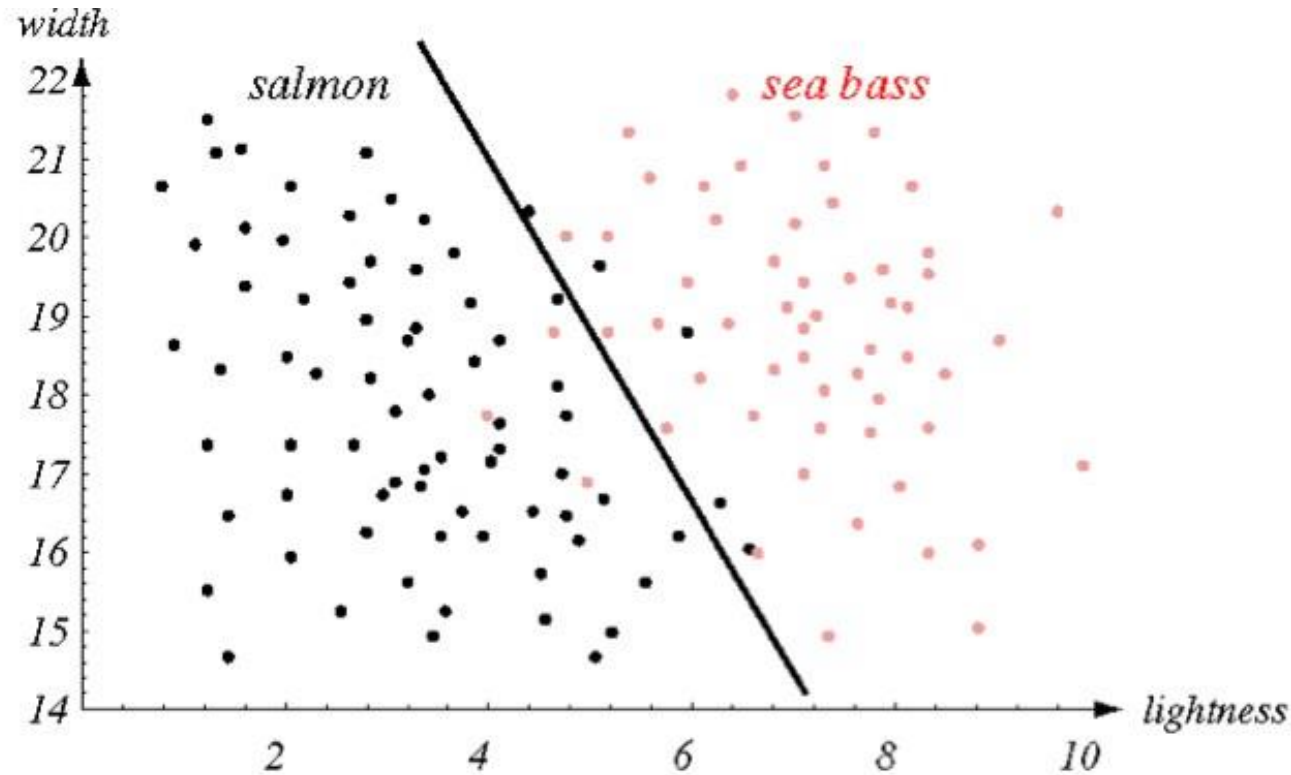


Figure 4: *Scatter plot* of lightness and width features for training samples. We can draw a *decision boundary* to divide the feature space into two regions. Does it look better than using only lightness?

An Example: Multiple Features

- Does adding more features always improve the results?
 - Avoid unreliable features.
 - Be careful about correlation with existing features.
 - Be careful about measurement costs.
 - Be careful about noise in the measurements.
- Is there some *curse* for working in very high dimensions?

Pattern Recognitions Systems

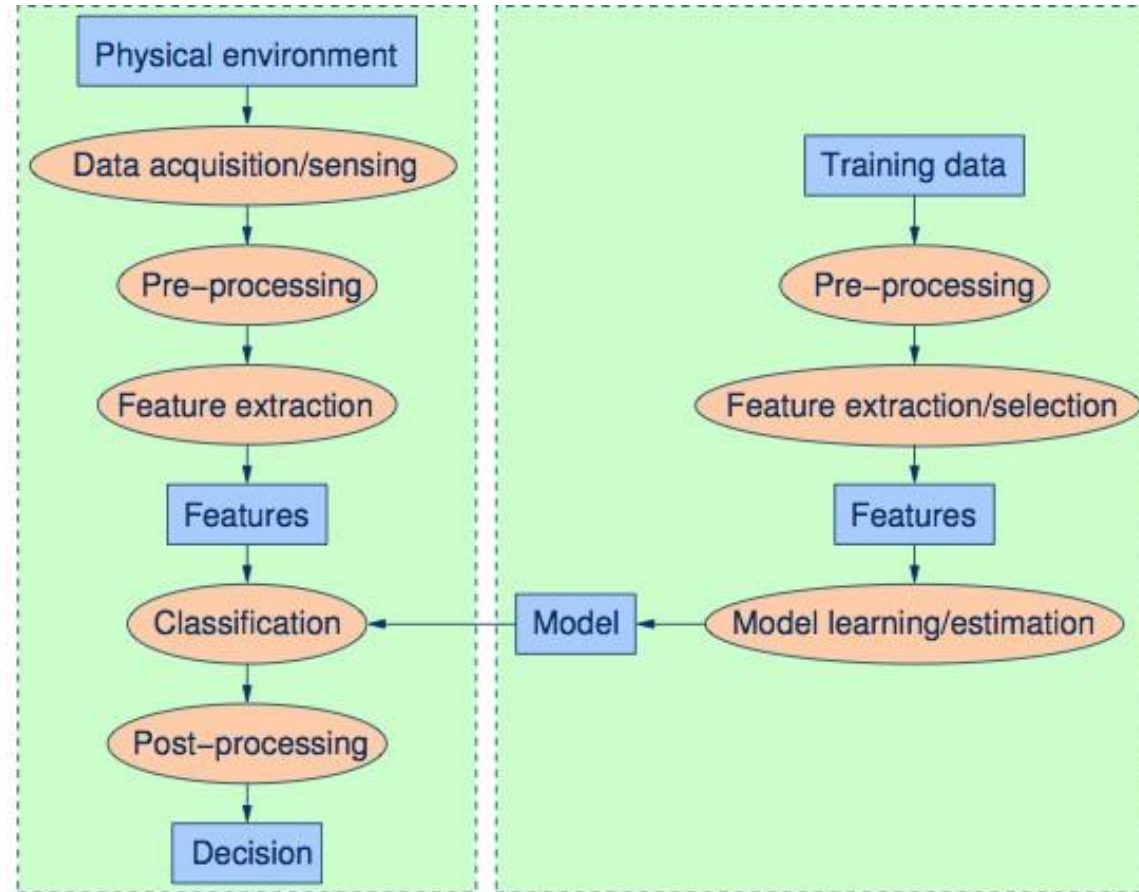


Figure 7: Object/process diagram of a pattern recognition system.

Pattern Recognitions Systems

- Data acquisition and sensing
 - Measurements of physical variables.
 - Important issues: bandwidth, resolution, sensitivity, distortion, SNR, latency, etc.
- Pre- processing
 - Removal of noise in data
 - Isolation of patterns of interest from the background
- Feature extraction
 - Finding a new representation in terms of features

Pattern Recognitions Systems

- Model learning and estimation
 - Learning a mapping between features and patterns groups and categories
- Classification
 - Using features and learned models to assign a pattern to a category.
- Post processing
 - Evaluation of confidence in decisions
 - Exploitation of context to improve performance
 - Combination of experts

The Design Cycle

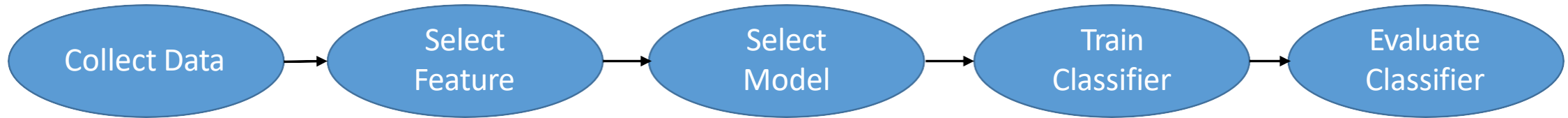


Figure 8 : The Design Cycle

Data Collection

- Collecting training and testing data
- How can we know when we have adequately large and representative set of samples?

The Design Cycle

Feature Selection

- Domain dependence and prior information
- Computational cost and feasibility
- Discriminative features
 - Similar values for similar patterns
 - Different values for different patterns
- Invariant features with respect to translation, rotation and scale
- Robust features with respect to occlusion, distortion, deformation, and variations in environment

The Design Cycle

Model Selection

- Domain dependence and prior information
- Definition of design criteria
- Parametric vs. non-parametric models.
- Handling of missing features
- Computational complexity
- Types of models: templates, decision-theoretic or statistical, syntactical or structural, neural, and hybrid
- How can we know how close we are to the true model underlying the patterns?

The Design Cycle

Training

- How can we learn the rule from data?
- Supervised learning:
 - A teacher provides a category label or cost for each pattern in the training set.
- Unsupervised learning:
 - The system forms clusters or natural grouping of the inputs patterns.
- Reinforcement learning:
 - No desired category is given but the teacher provides feedback to the system such as decision is right or wrong.

The Design Cycle

Evaluation

- How can we estimate the performance with training samples?
- How can we predict the performance with future data?
- Problems of over-fitting and generalization.

Statistical Pattern Classification Techniques

- We have many statistical pattern classification techniques. In this section, we discuss three basic techniques:
 - The minimum distance classifier
 - The k-nearest neighbors (KNN) classifier, and
 - The maximum likelihood (or Bayesian) classifier.
- We know that objects' properties can be represented using *feature vectors* that are projected onto a *feature space*.
- Feature vectors associated with objects from the same class will appear close together as *clusters* in the feature space.
- The job of a statistical pattern classification techniques *is to find a discrimination curve* (or a *hypersurface*, in the case of an n -dimensional feature space) *that can tell the clusters apart*.

Classifier concept

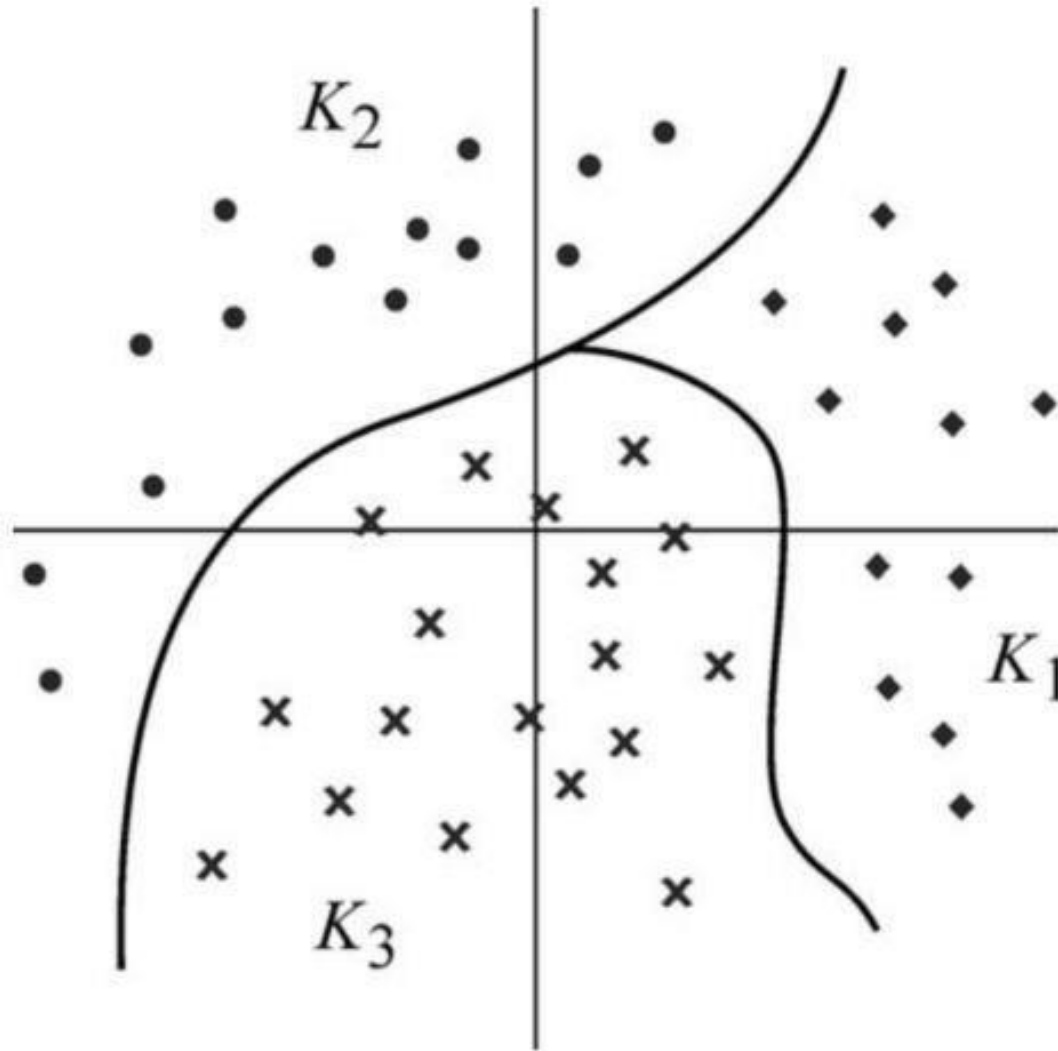


Figure: Discrimination functions for a three-class classifier in a 2D feature space

Statistical Pattern Classification Techniques

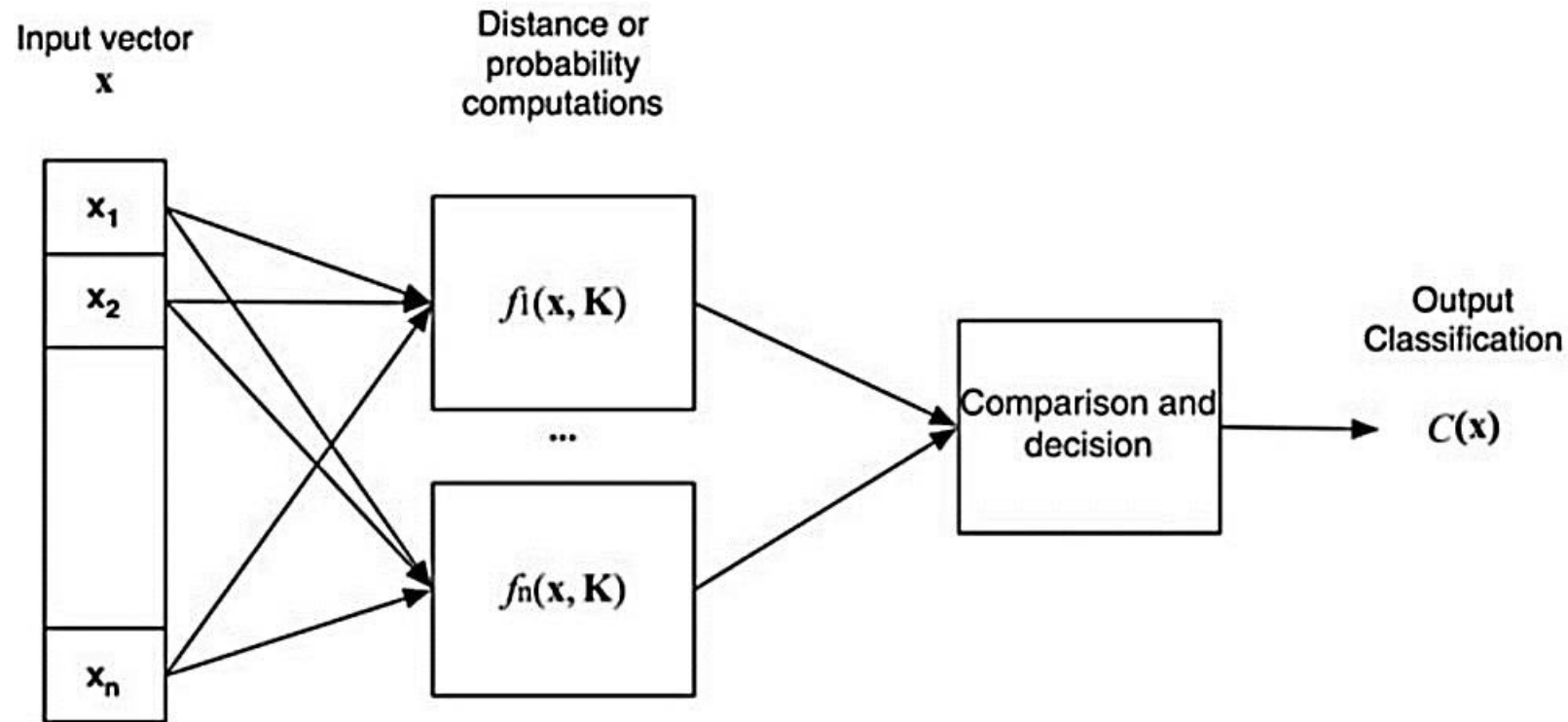


Figure: Diagram of a Statistical pattern Classifier

Statistical Pattern Classification Techniques

- A statistical pattern classifier has n inputs (the features of the object to be classified encoded into feature vector , $\mathbf{x} = x_1, x_2, x_3 \dots, x_n)^T$ and one output (the class to which the object belongs , $C(\mathbf{x})$, are represented by one of the symbols by $w_1, w_2, w_3, \dots, w_W$ where W is the number of classes). The symbols w_W are *class identifiers*.
- Classifiers make comparisons among the representations of the unknown object and the known classes. These comparisons provide information to make decision about which class to assign to the unknown pattern.
- Mathematically, the job of the classifier is to apply a series of *decision rules* that divide the feature space into W disjoint subsets K_w , $w = 1, 2, \dots, W$, each of which includes the feature vectors $\mathbf{x}$ for which $d(\mathbf{x}) = w_w$, where $d(.)$ is the *decision rule*.

Minimum Distance Classifier

- The minimum distance classifier (also known as the nearest-class mean classifier) works by computing a distance metric between an unknown feature vector and the centroids (i.e. mean vectors) of each class:

$$d_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\|,$$

- where d_j is a distance metric between class j and the unknown feature vector $\mathbf{x}$, $\mathbf{m}_j$ is the mean vector for class j , defined as

$$\mathbf{m}_j = \frac{1}{N_j} \sum_{\mathbf{x} \in w_j} \mathbf{x}_j$$

- Where N_j is the number of pattern vectors from class w_j .

Minimum Distance Classifier

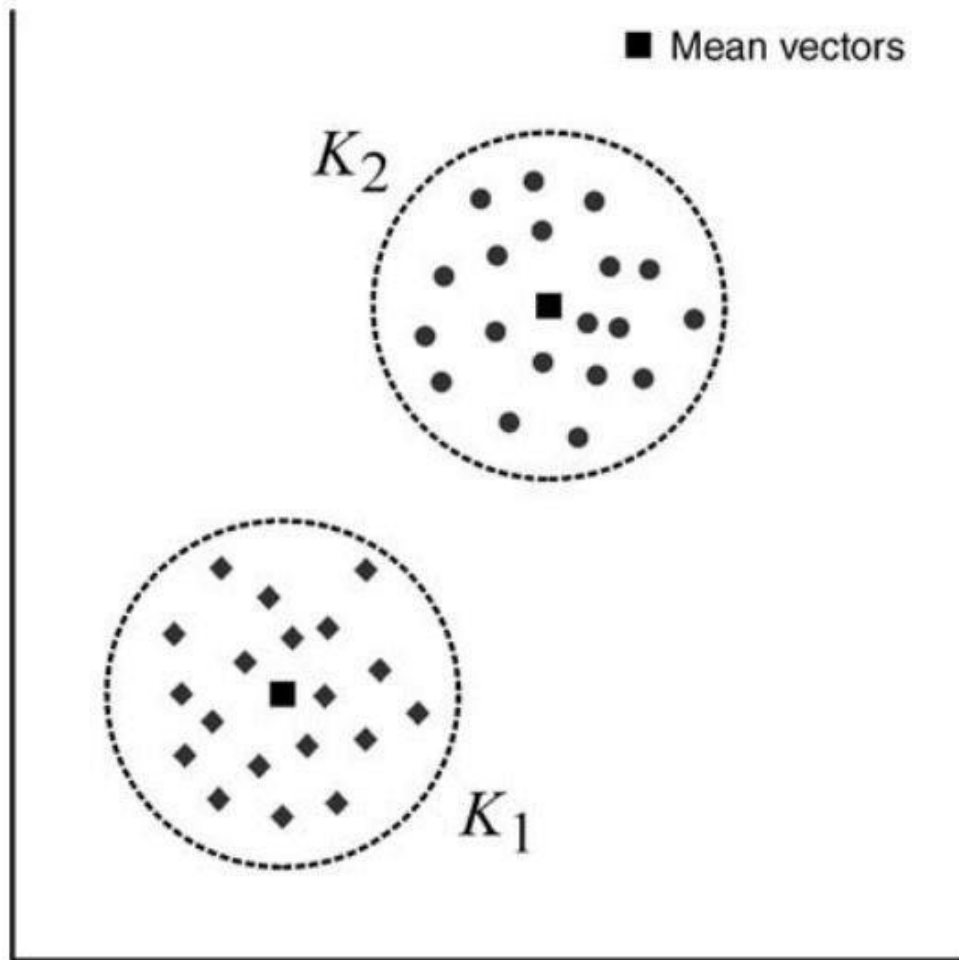


Figure: Example of two classes and their mean vectors.

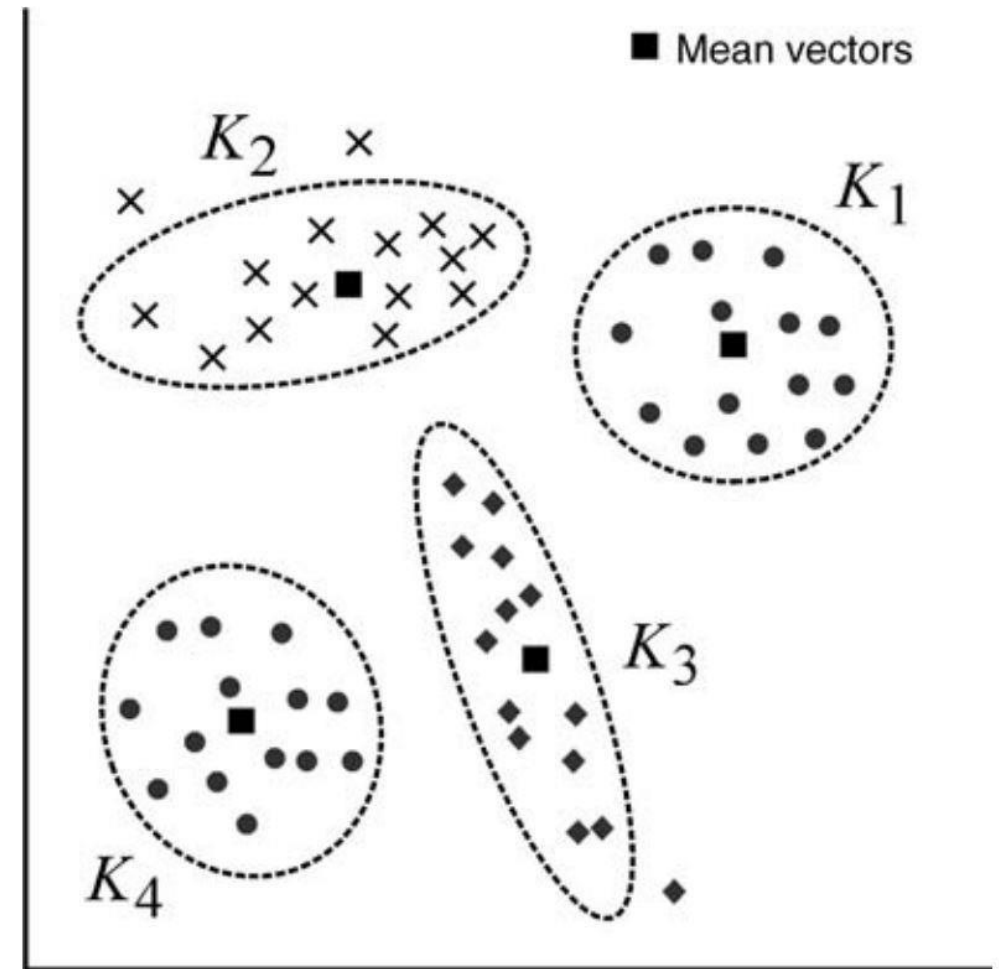


Figure: Example of three classes with relatively complex structure

K-Nearest Neighbors Classifier

- The k-nearest neighbours (KNN) classifier works by computing the distance between an unknown pattern's feature vector $\mathbf{x}$ and the closest points to it in the feature space, and then assigning the unknown pattern to the class to which the majority of the k samples points belong.
- The **main advantages** of this approach are its **simplicity** (e.g., no assumptions need to be made about the probability distribution of each class) and **versatility** (e.g., it handles overlapping classes or classes with complex structure as well).
- Its **main disadvantage** is the **computational cost** involved in computing distances between the unknown sample and many stored points in the feature space.

K-Nearest Neighbors Classifier

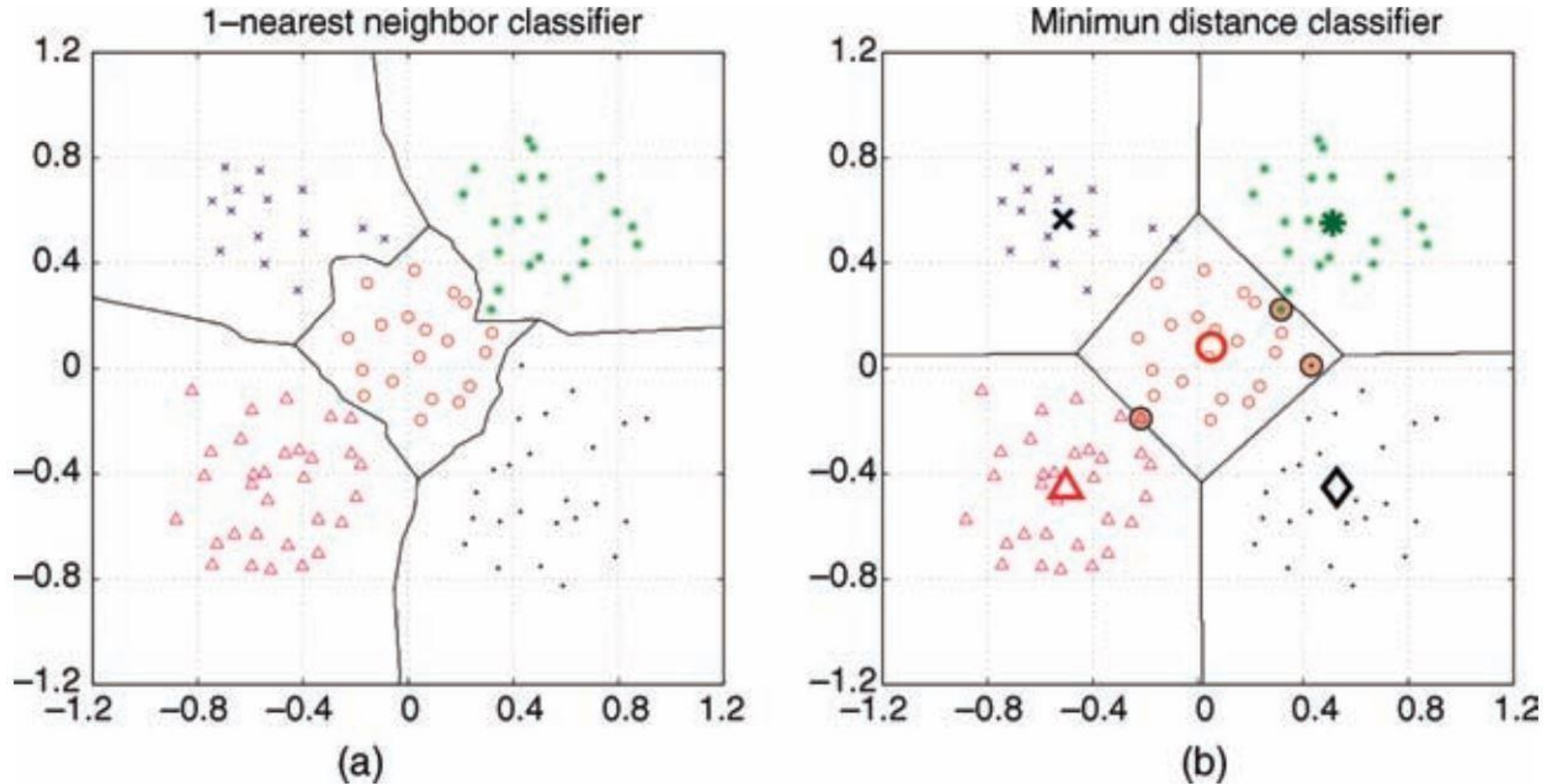


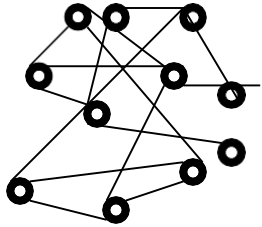
Figure: (a) Example of KNN classifier ($k = 1$) for a five-class classifier in a 2D feature space. (b) Minimum distance classifier results for the same data set.

Bayesian Classifier

- The Bayesian Classifier is that a classification decision can be made based on the *probability distributions* of the training samples for each class; that is, an unknown object is assigned to the class to which it is *more likely* to belong based on the observed features.
- The mathematical calculations performed by a Bayesian classifier require three probability distributions:
 - The *a priori* (or *prior*) probability for each w_k , denoted by $P(w_k)$.
 - The *unconditional distribution* of the feature vector representing the measured pattern $\mathbf{x}$, denoted by $p(\mathbf{x})$.
 - The *class conditional distribution*, that is, the probability of $\mathbf{x}$ given class w_k , denoted by $p(\mathbf{x} | w_k)$.
- Applying Bayes' rule, these three distributions are then used to compute the a *posteriori probability* that a pattern $\mathbf{x}$ comes from class w_k , represented as $p(w_k | \mathbf{x})$, as follows:

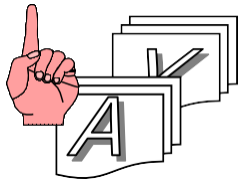
$$p(w_k | \mathbf{x}) = \frac{p(\mathbf{x} | w_k) \cdot P(w_k)}{p(\mathbf{x})} = \frac{p(\mathbf{x} | w_k) \cdot P(w_k)}{\sum_{k=1}^W p(\mathbf{x} | w_k) \cdot P(w_k)}$$

Overview of Neural Network in image processing



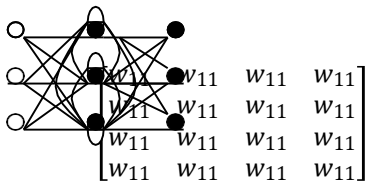
Connectionizm

NN is a highly interconnected structure in such a way that the state of one neuron affects the potential of the large number of another neurons to which it is connected according to weights of connections



Not Programming but Training

NN is trained rather than programmed to perform the given task since it is difficult to separate the hardware and software in the structure. We program not solution of tasks but ability of learning to solve the tasks



Distributed Memory

NN presents an distributed memory so that changing-adaptation of synapse can take place everywhere in the structure of the network.



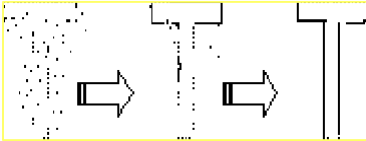
Learning and Adaptation

NN are capable to adapt themselves (the synapses connections between units) to special environmental conditions by changing their structure or strengths connections.

$$y = \phi(\sqrt{x^2})$$

Non-Linear Functionality

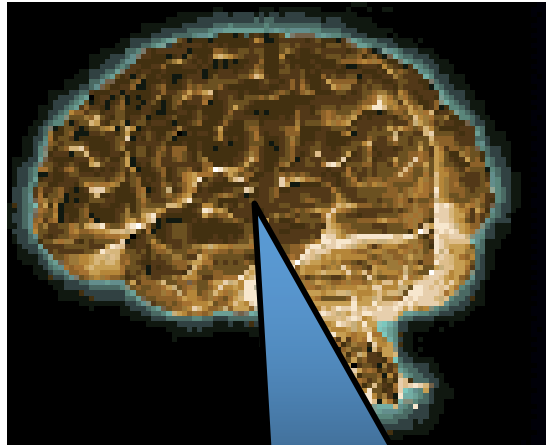
Every new states of a neuron is a nonlinear function of the input pattern created by the firing nonlinear activity of the other neurons.



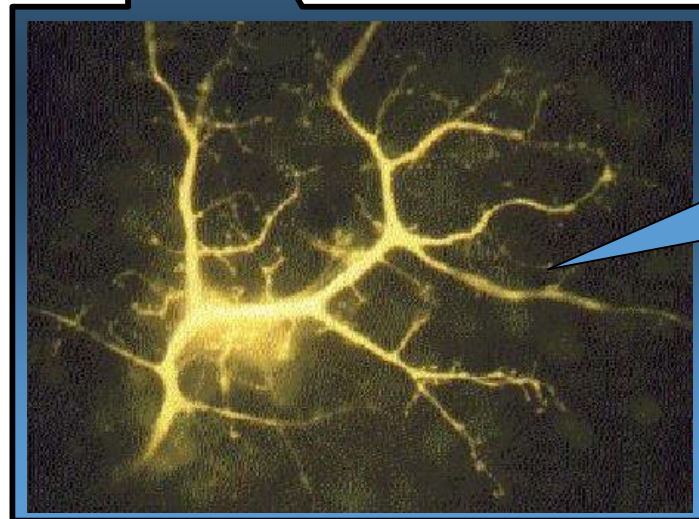
Robustness of Associativity

NN states are characterized by high robustness or insensitivity to noisy and fuzzy of input data owing to use of a highly redundance distributed structure

Brain Computer: What is it?



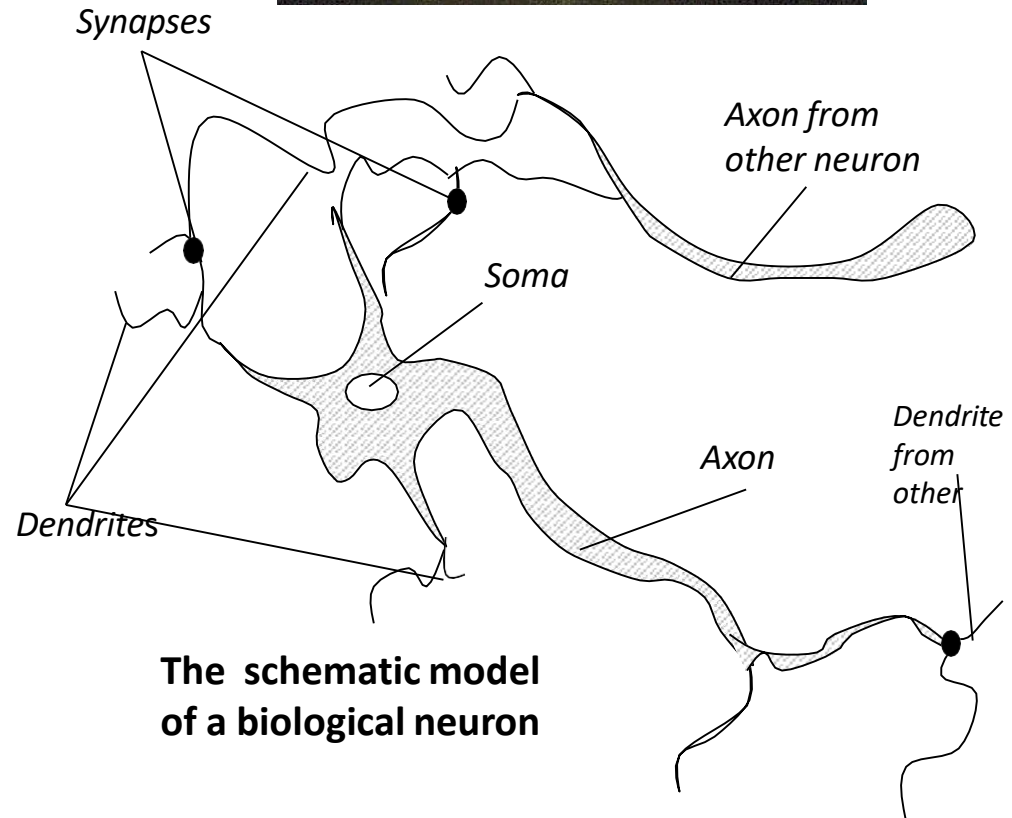
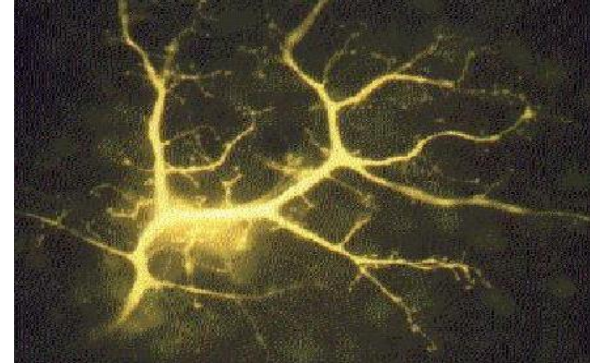
*Human brain contains
a massively
interconnected net of
 10^{10} - 10^{11} neurons or
nerve cells*



Biological Neuron
- The simple
arithmetic
computing"
element

Biological Prototypes and Artificial Neurons

1. **Soma or body cell** - is a large, round central body in which almost all the logical functions of the neuron are realized.
2. **The axon (output)**, is a nerve fibre attached to the soma which can serve as a final output channel of the neuron. An axon is usually highly branched.
3. **The dendrites (inputs)** - represent a highly branching tree of fibres. These long irregularly shaped nerve fibres (processes) are attached to the soma.
4. **Synapses** are specialized contacts on a neuron which are the termination points for the axons from other neurons.



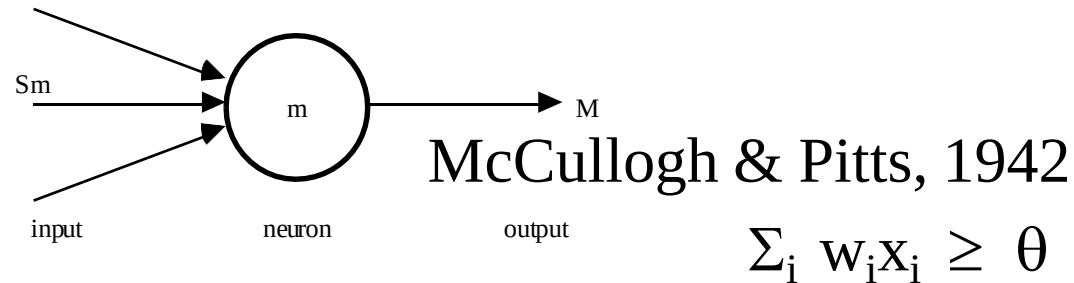
Artificial Neural Networks

Artificial Neural Networks provide...

- A new computing paradigm
- A technique for developing trainable classifiers, memories, dimension-reducing mappings, etc
- A tool to study brain function

Vision, AI and ANNs

- 1940s: beginning of Artificial Neural Networks



Perceptron learning rule (Rosenblatt, 1962)

Backpropagation

Hopfield networks (1982)

Kohonen self-organizing maps

Vision, AI and ANNs

1950s: beginning of computer vision

Aim: *give to machines same or better vision capability as ours*

Drive: AI, robotics applications and factory automation

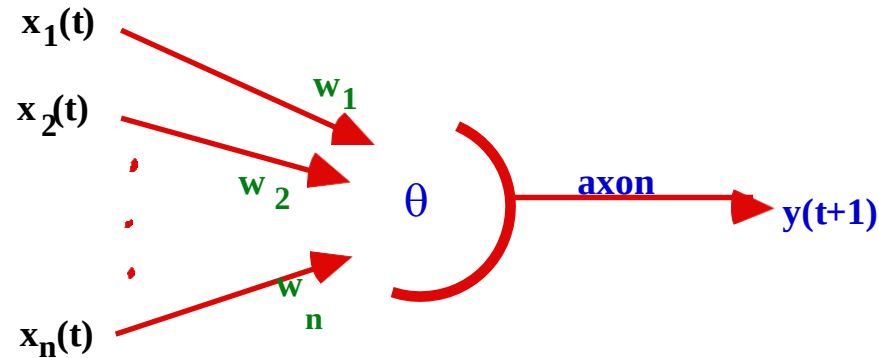
Initially: passive, feedforward, layered and hierarchical process
that was just going to provide input to higher reasoning
processes (from AI)

But soon: realized that could not handle real images

1980s: Active vision: make the system more robust by allowing the
vision to adapt with the ongoing recognition/interpretation

Warren McCulloch and Walter Pitts (1943)

- A McCulloch-Pitts neuron operates on a discrete time-scale, $t = 0, 1, 2, 3, \dots$ with time tick equal to one refractory period



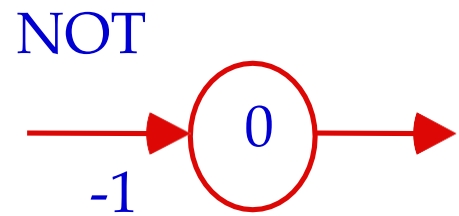
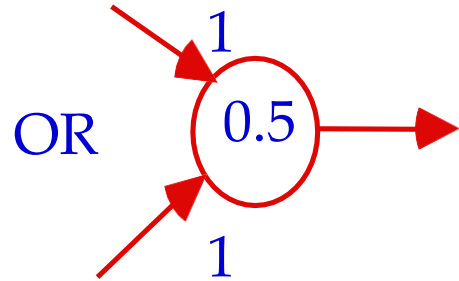
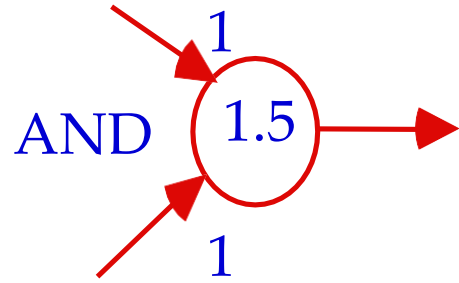
- At each time step, an input or output is *on* or *off* — 1 or 0, respectively.
- Each connection or synapse from the output of one neuron to the input of another, has an attached weight.

Excitatory and Inhibitory Synapses

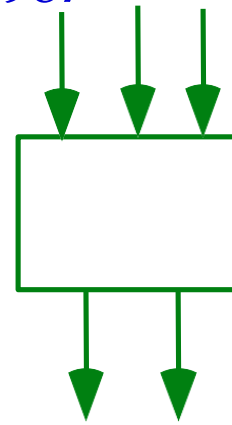
- We call a synapse
 excitatory if $w_i > 0$, and
 inhibitory if $w_i < 0$.
- We also associate a **threshold** θ with each neuron
- A neuron fires (i.e., has value 1 on its output line) at time $t+1$ if the weighted sum of inputs at t reaches or passes θ :

$$y(t+1) = 1 \quad \text{if and only if} \quad \sum w_i x_i(t) \geq \theta$$

From Logical Neurons to Finite Automata

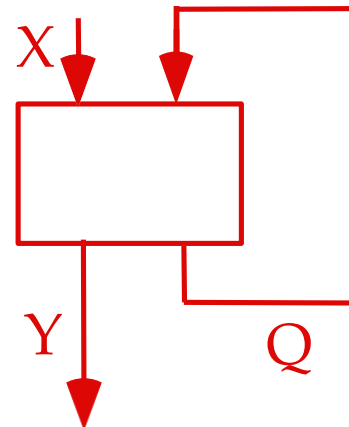


Brains, Machines, and
Mathematics, 2nd Edition,
1987



Boolean Net

$X \rightarrow Y$



Finite
Automaton

Hopfield Networks

- A paper by John Hopfield in 1982 was the catalyst in attracting the attention of many physicists to "Neural Networks".
- In a network of McCulloch-Pitts neurons whose output is 1 iff $\sum w_{ij} s_j \geq \theta_i$ and is otherwise 0, neurons are updated synchronously: every neuron processes its inputs at each time step to determine a new output.

Hopfield Networks

- A Hopfield net (Hopfield 1982) is a net of such units subject to the **asynchronous rule for updating one neuron at a time**:

"Pick a unit i at random.

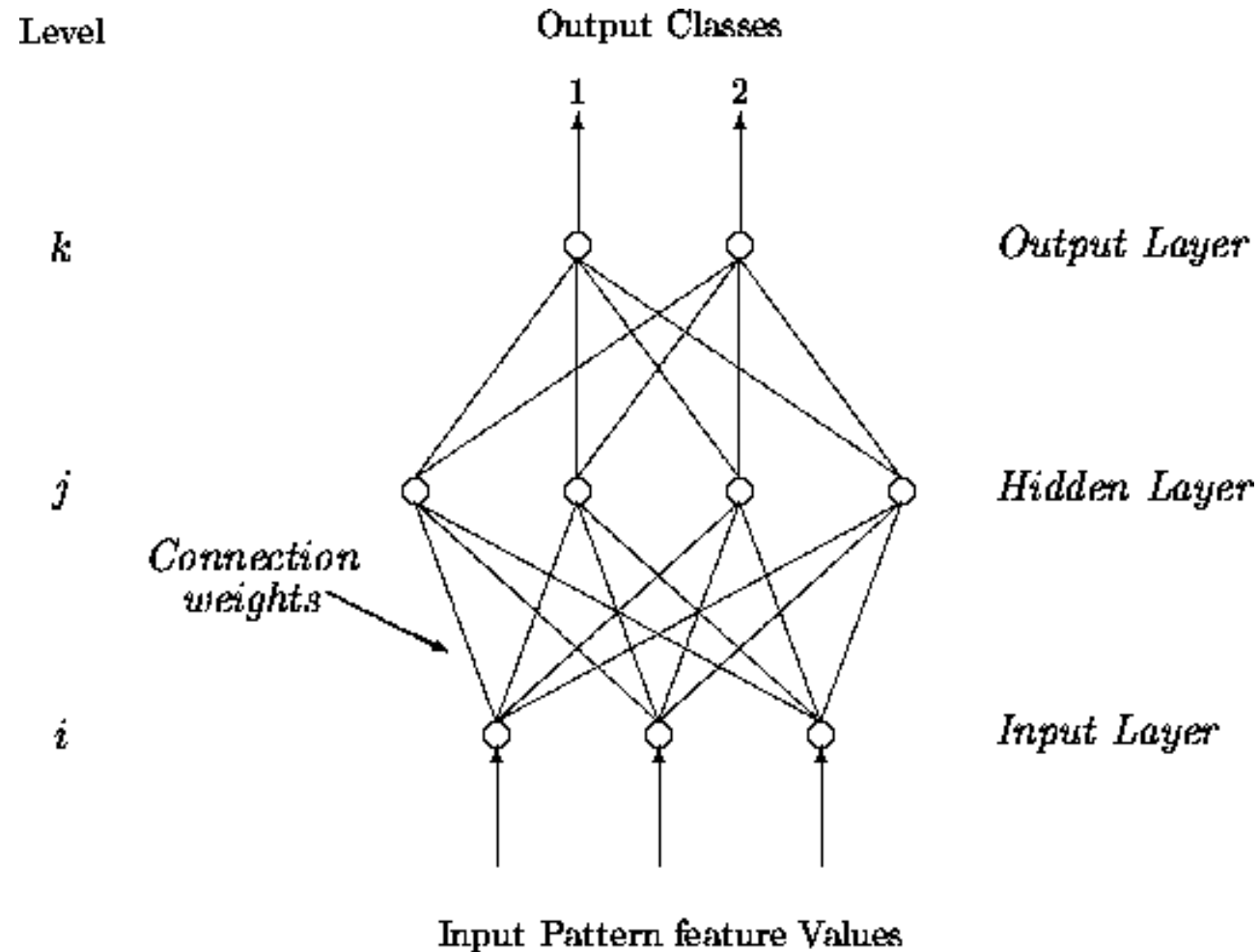
If $\sum w_{ij} s_j \geq \theta_i$, turn it on.

Otherwise turn it off."

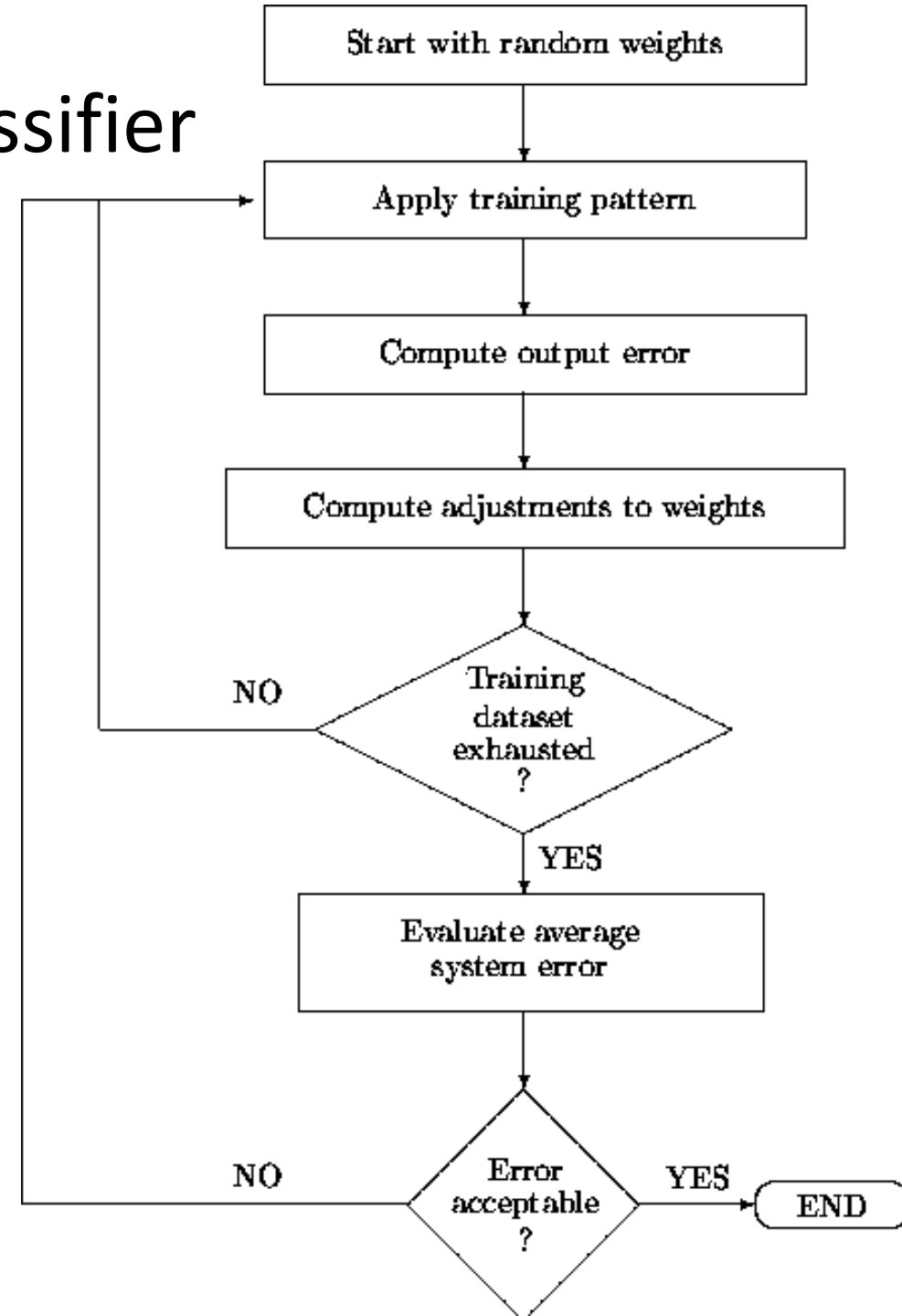
- Moreover, Hopfield assumes **symmetric weights**:

$$w_{ij} = w_{ji}$$

Multi-layer Perceptron Classifier

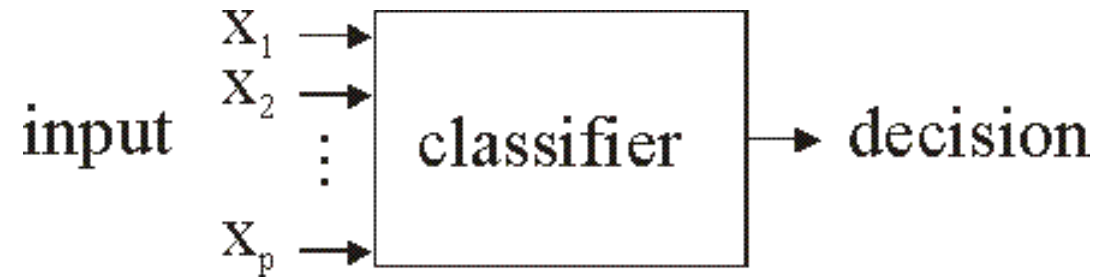


Multi-layer Perceptron Classifier

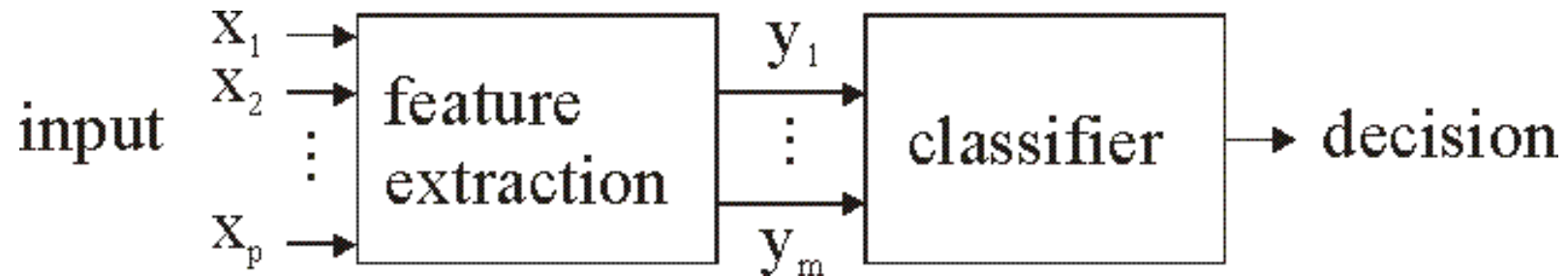


Classifiers

- 1-Stage approach

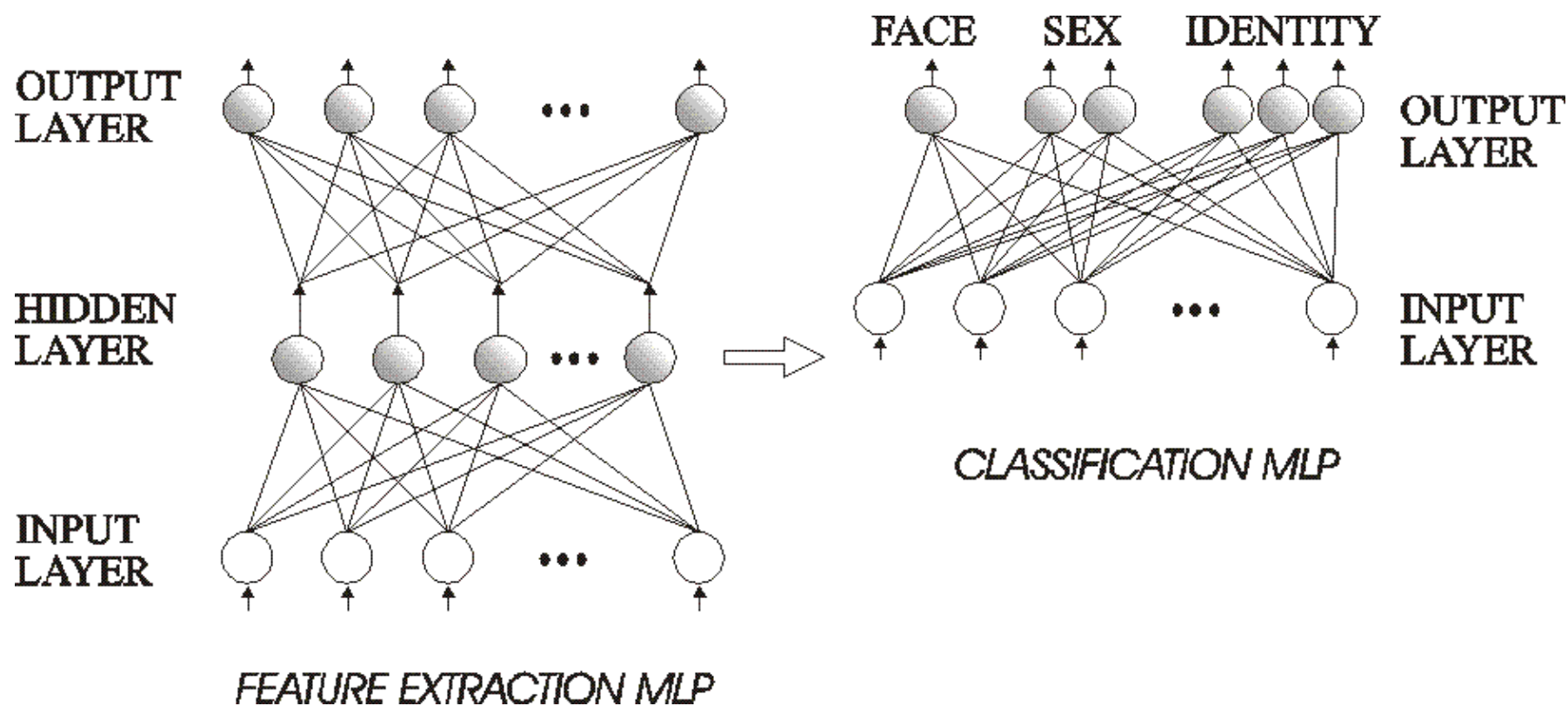


- 2-Stage approach



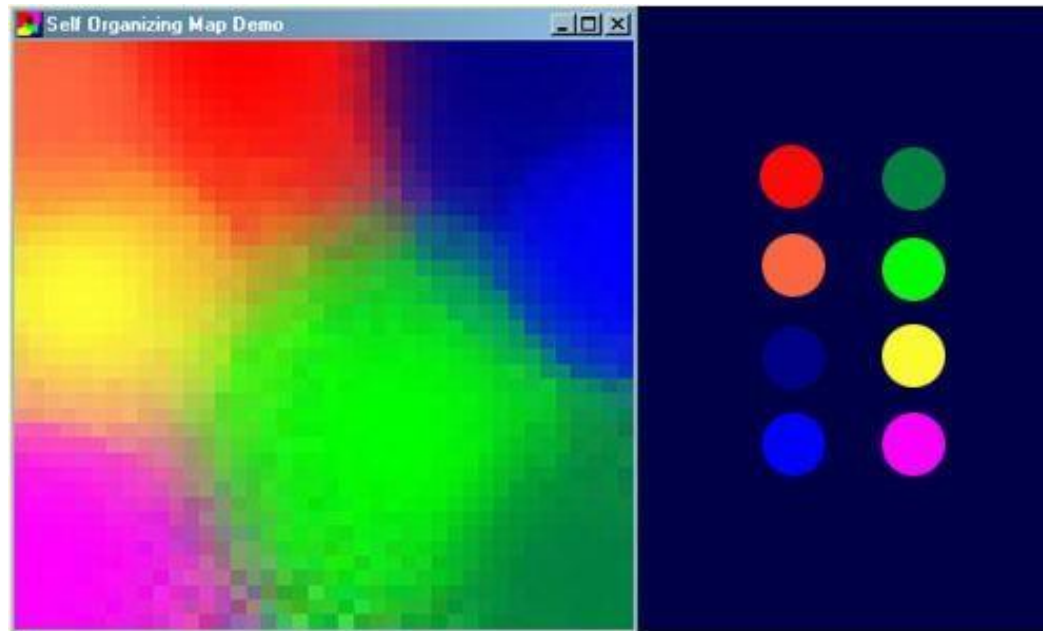
Example: face recognition

- Here using the 2-stage approach:



SOM : Self Organized Map

- Provide a way of representing multidimensional data in much lower dimensional spaces - usually one or two dimensions.
- Vector quantization : Reducing the dimensionality of vectors.



Nuro Fuzzy Systems

Neural Networks

Neural networks are good at recognizing patterns, they are not good at explaining how they reach their decisions.

Fuzzy Systems

Fuzzy logic systems, which can reason with imprecise information, are good at explaining their decisions but they cannot automatically acquire the rules they use to make those decisions.

Need a Intelligent Hybrid System !!